

资产管理系统 — 架构与设计概要

文档版本：v1.0

编写日期：2026-06-10

文档性质：架构设计文档（SAD，Software Architecture Document）

关联文档：需求文档 v1.0、设计概要 v1.0

目录

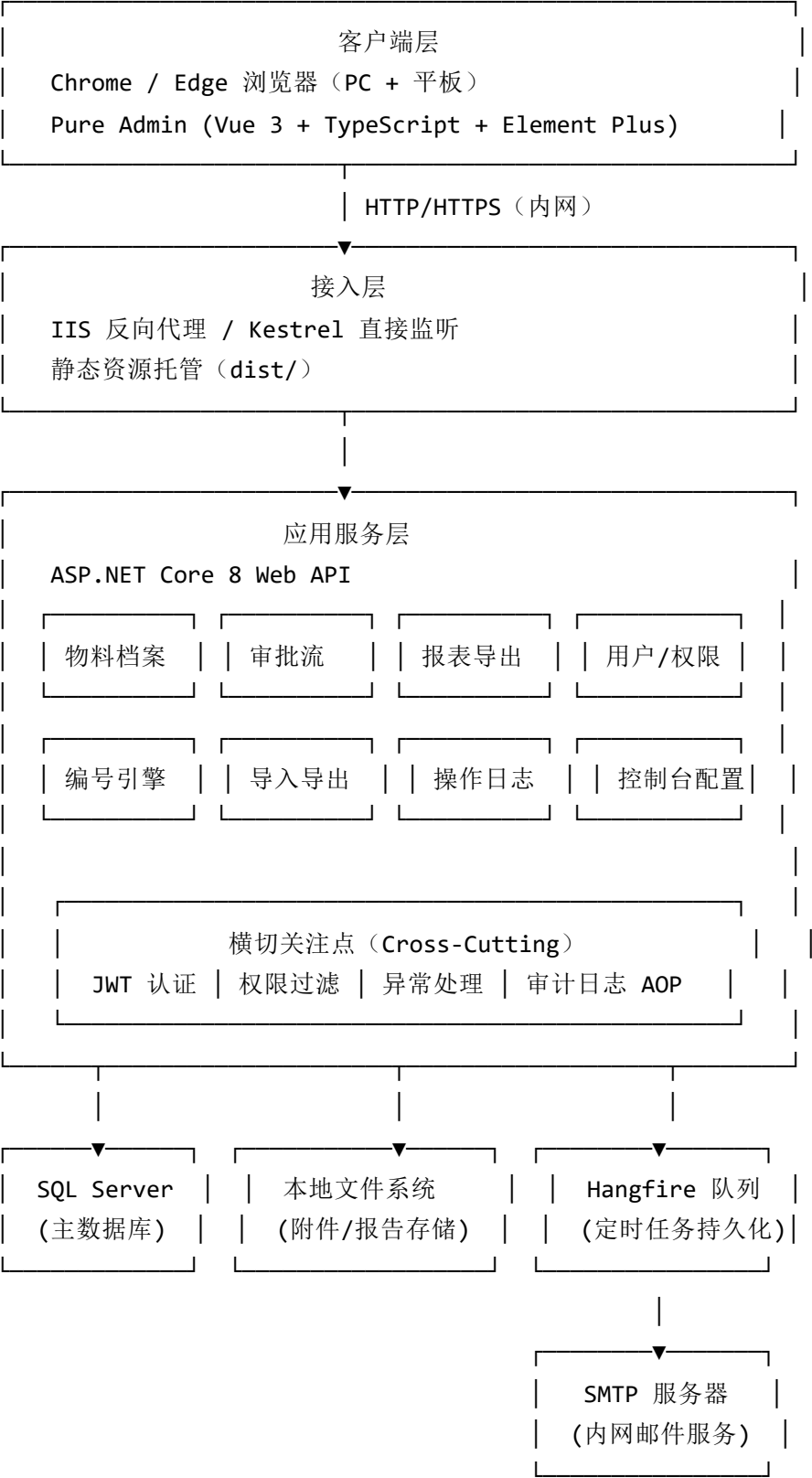
1. 架构总览
2. 分层架构设计
3. 后端模块划分
4. 前端模块划分
5. 数据流与核心时序
6. 关键子系统设计
7. 安全架构设计
8. 部署架构
9. 关键技术决策记录

1. 架构总览

1.1 系统定位

单一部门内网 Web 应用，**单体架构（Monolith）**，无需微服务拆分。用户规模小（预估并发 < 50 人），业务边界清晰，优先保证开发效率和运维简单性。

1.2 整体架构图



1.3 核心架构决策

决策	选择	理由
整体架构	单体（Monolith）	用户规模小，单进程部署运维成本最低
前后端关系	分离（SPA + API）	前端独立开发部署，接口可复用
部署单元	单一进程（含 Hangfire）	无需容器，IIS 或 Kestrel 直接托管
数据库	单库单实例	数据量不大，无读写分离必要
异步任务	In-Process Hangfire	不引入 MQ，Hangfire 持久化足够可靠

2. 分层架构设计

2.1 后端分层结构

```
src/
├─ AssetManagement.Api/           # 表现层：Controller、Middleware、Filter
├─ AssetManagement.Application/   # 应用层：Service、Command/Query、DTO、Validator
├─ AssetManagement.Domain/        # 领域层：Entity、枚举、领域规则、接口定义
├─ AssetManagement.Infrastructure/ # 基础设施层：EF Core、邮件、文件、Hangfire
└─ AssetManagement.Shared/        # 公共层：常量、扩展方法、工具类
```

2.2 各层职责说明

表现层（Api）

负责 HTTP 请求的接收与响应，不含业务逻辑。

Controllers/

— AuthController.cs	# 登录、注销、刷新 Token
— AssetController.cs	# 物料档案 CRUD、状态变更、附件
— CategoryController.cs	# 类别树、编号规则配置
— LocationController.cs	# 位置树、二维码下载
— ApprovalController.cs	# 审批工单、归还确认、代审配置
— ImportController.cs	# Excel 导入（预校验 + 确认）
— ReportController.cs	# 统计报表、Excel 导出
— AuditLogController.cs	# 操作日志查询
— AdminController.cs	# 控制台：用户、参数、邮件配置

Middleware/

— ExceptionHandlingMiddleware.cs	# 全局异常捕获，统一错误响应格式
— RequestLoggingMiddleware.cs	# 请求日志（记录 trace_id）

Filters/

— PermissionFilter.cs	# 基于角色的权限校验（ActionFilter）
— AuditLogFilter.cs	# 写操作自动记录审计日志（ActionFilter）

应用层（Application）

编排业务流程，调用领域对象和基础设施，返回 DTO。

Assets/

- └─ Commands/
 - | └─ CreateAssetCommand.cs # 新增物料（含编号生成）
 - | └─ UpdateAssetCommand.cs # 编辑物料
 - | └─ ChangeAssetStatusCommand.cs # 状态变更
- └─ Queries/
 - | └─ GetAssetListQuery.cs # 分页查询
 - | └─ GetAssetDetailQuery.cs # 详情（含流转记录）
- └─ Dtos/
 - | └─ AssetListDto.cs
 - | └─ AssetDetailDto.cs

Approvals/

- └─ Commands/
 - | └─ CreateApprovalCommand.cs # 发起审批
 - | └─ ApproveCommand.cs # 审批通过
 - | └─ RejectCommand.cs # 审批驳回
 - | └─ ConfirmReturnCommand.cs # 确认归还
- └─ Jobs/
 - | └─ AutoApproveTimeoutJob.cs # 超时自动审批（Hangfire 定时）
 - | └─ OverdueReminderJob.cs # 逾期提醒（Hangfire 定时）

NumberRules/

- └─ NumberGeneratorService.cs # 编号生成核心逻辑（乐观锁）

Import/

- └─ ImportValidateService.cs # Excel 预校验
- └─ ImportExecuteService.cs # 执行导入

Reports/

- └─ ReportQueryService.cs # 报表数据查询
- └─ ExcelExportService.cs # Excel 导出

领域层（Domain）

系统核心实体和业务规则，不依赖任何框架。

```
Entities/
├─ Asset.cs                # 物料（含状态流转方法）
├─ AssetCategory.cs        # 物料类别
├─ NumberRule.cs           # 编号规则
├─ Location.cs             # 位置
├─ User.cs                 # 用户（含代审有效期判断）
├─ ApprovalFlow.cs         # 审批流水（含超时判断方法）
├─ FlowRecord.cs           # 审批操作记录
├─ AuditLog.cs             # 操作日志
└─ SystemSetting.cs        # 系统参数
```

```
Enums/
├─ AssetStatus.cs          # 物料状态枚举
├─ FlowType.cs             # 审批类型枚举
├─ FlowStatus.cs           # 审批状态枚举
└─ UserRole.cs             # 用户角色枚举
```

```
Interfaces/
├─ IAssetRepository.cs      # 物料仓储接口
├─ IApprovalRepository.cs   # 审批仓储接口
├─ IEmailService.cs         # 邮件服务接口
├─ IFileStorageService.cs   # 文件存储接口
└─ INumberGenerator.cs      # 编号生成接口
```

基础设施层（Infrastructure）

接口的具体实现，包含所有外部依赖。

```
Persistence/
├─ AppDbContext.cs         # EF Core DbContext
├─ Migrations/             # EF Core 迁移文件
└─ Repositories/           # 各实体仓储实现

Services/
├─ MailKitEmailService.cs   # SMTP 邮件发送（MailKit）
├─ LocalFileStorageService.cs # 本地文件读写
└─ HangfireJobScheduler.cs  # 定时任务注册

Excel/
├─ EpPlusExcelReader.cs     # Excel 导入读取
└─ EpPlusExcelWriter.cs     # Excel 导出生成
```

2.3 依赖方向

Api → Application → Domain

Infrastructure → Domain（实现接口）

Api → Infrastructure（DI 注入）

领域层不依赖任何外层，保持纯净。

3. 后端模块划分

3.1 编号生成模块

编号生成是并发敏感的核心逻辑，独立封装。

输入：category_id

输出：asset_no（如 PLC-MIT-Q-004）

流程：

1. 读取 number_rule（含 segment_def 和 seq_current）
2. 解析 segment_def，拼接固定段
3. 使用乐观锁更新 seq_current：
UPDATE number_rule
SET seq_current = seq_current + 1, updated_at = GETDATE()
WHERE id = @id AND seq_current = @expected
4. 若更新行数为 0（并发冲突），重试最多 3 次
5. 拼接流水号段（补零至配置位数）
6. 返回完整编号

关键约束：

- 编号生成和物料记录写入在同一事务内
- 乐观锁失败重试间隔 50ms，最多 3 次，仍失败返回 409

3.2 审批流模块

审批流状态机由 ApprovalFlow 实体内部管理，外部只能调用方法，不能直接改状态字段。

```
// 领域实体中的状态流转方法（伪代码示意）
public class ApprovalFlow
{
    public FlowStatus Status { get; private set; }

    public void Approve(int actualApproverId, string comment)
    {
        if (Status != FlowStatus.Pending)
            throw new DomainException("只有待审批状态才能操作通过");
        Status = FlowStatus.Approved;
        ActualApproverId = actualApproverId;
        ApproveTime = DateTime.Now;
        AddRecord(FlowAction.Approve, actualApproverId, comment);
    }

    public void Reject(int actualApproverId, string reason)
    {
        if (Status != FlowStatus.Pending)
            throw new DomainException("只有待审批状态才能操作驳回");
        Status = FlowStatus.Rejected;
        RejectReason = reason;
        AddRecord(FlowAction.Reject, actualApproverId, reason);
    }

    public void AutoApproveOnTimeout()
    {
        if (Status != FlowStatus.Pending || Deadline > DateTime.Now)
            return;
        Status = FlowStatus.AutoApproved;
        AddRecord(FlowAction.AutoApprovedTimeout, operatorId: null, "超时自动通过");
    }
}
```

3.3 审计日志模块

审计日志通过 **ActionFilter** 切面实现，业务代码无需手动调用，做到零侵入。

执行顺序：

Request → PermissionFilter → Controller Action → AuditLogFilter → Response

AuditLogFilter 逻辑：

OnActionExecuting: 记录操作前状态（UPDATE 类型读取实体快照）

OnActionExecuted:

- └─ 响应成功（2xx）→ 计算 diff，异步写入 audit_log
- └─ 响应失败 → 不写日志（失败操作无需审计）

diff 计算策略：

- CREATE: diff = null, summary 写"新增物料 PLC-MIT-Q-004"
- UPDATE: diff = { field: {before, after} }, 只记录有变化的字段
- DELETE/STATUS_CHANGE: diff = null, summary 描述变更内容

写入方式：异步（Task.Run），不阻塞主请求响应

3.4 定时任务模块

所有定时任务通过 Hangfire 管理，持久化到数据库，服务重启不丢任务。

任务一：超时自动审批扫描

- Cron: 每 15 分钟执行一次
- 逻辑: `SELECT * FROM approval_flow WHERE status='pending' AND deadline < NOW()`
- 对每条记录调用 `AutoApproveOnTimeout()`, 发送邮件通知

任务二：逾期提醒

- Cron: 每天 09:00 执行 (由 `system_setting` 配置)
- 逻辑: `SELECT * FROM approval_flow
JOIN asset ON ... WHERE status='approved'
AND expected_return_date < TODAY()
AND asset.status = 'borrowed'`
- 对每条记录发送逾期提醒邮件 (保管人 + 主管)

任务三：到期前提醒

- Cron: 每天 09:00 执行 (与逾期提醒同批次)
- 逻辑: `expected_return_date = TODAY() + remind_days_before`
- 发送归还提醒邮件给当前保管人

任务四：审计日志清理

- Cron: 每月 1 日 02:00 执行
- 逻辑: `DELETE FROM audit_log
WHERE occurred_at < DATEADD(MONTH, -@retention_months, NOW())`

4. 前端模块划分

4.1 目录结构

```
src/
├── api/                                # API 请求封装 (axios)
│   ├── assets.ts
│   ├── approvals.ts
│   ├── reports.ts
│   └── admin.ts
│
├── stores/                             # Pinia 状态管理
│   ├── auth.ts                        # 当前用户、Token、权限
│   ├── category.ts                   # 类别树缓存
│   └── location.ts                   # 位置树缓存
│
├── router/
│   └── index.ts                       # 路由配置 (含权限守卫)
│
├── views/                              # 页面级组件
│   ├── auth/
│   │   └── LoginView.vue
│   ├── assets/
│   │   ├── AssetListView.vue
│   │   ├── AssetDetailView.vue
│   │   ├── AssetFormView.vue
│   │   └── AssetImportView.vue
│   ├── approvals/
│   │   ├── PendingView.vue
│   │   ├── MineView.vue
│   │   └── CompletedView.vue
│   ├── reports/
│   │   ├── SummaryView.vue
│   │   ├── BorrowedView.vue
│   │   └── OverdueView.vue
│   └── admin/
│       ├── UserManageView.vue
│       ├── CategoryManageView.vue
│       ├── LocationManageView.vue
│       ├── SettingsView.vue
│       └── AuditLogView.vue
```

- └─ components/ # 可复用组件
 - └─ CategoryTreeSelect.vue # 类别树形选择器
 - └─ LocationTreeSelect.vue # 位置树形选择器
 - └─ StatusTag.vue # 物料状态标签（带颜色）
 - └─ ApprovalCard.vue # 审批工单卡片
 - └─ FlowTimeline.vue # 审批流转时间线
 - └─ NumberRuleEditor.vue # 编号规则段配置器
- └─ composables/ # 可复用逻辑（Vue Composable）
 - └─ usePermission.ts # 权限判断 hook
 - └─ useDownload.ts # 文件下载（Excel/ZIP）
 - └─ usePagination.ts # 分页逻辑封装
- └─ utils/
 - └─ http.ts # axios 实例（含拦截器）
 - └─ auth.ts # Token 存取
 - └─ format.ts # 日期、金额格式化

4.2 路由权限守卫

// router/index.ts (关键逻辑示意)

```
router.beforeEach(async (to) => {
  const authStore = useAuthStore()

  // 1. 未登录 → 跳转登录页
  if (!authStore.token && to.path !== '/login') {
    return '/login'
  }

  // 2. 路由元数据定义所需角色
  const requiredRoles = to.meta.roles as string[] | undefined
  if (requiredRoles && !requiredRoles.includes(authStore.user.role)) {
    return '/403' // 无权限页面
  }
})

// 路由定义示例
{
  path: '/admin/users',
  component: UserManageView,
  meta: { roles: ['admin'] }
}
{
  path: '/reports/summary',
  component: SummaryView,
  meta: { roles: ['warehouse', 'supervisor', 'admin'] }
}
```

4.3 全局 HTTP 拦截器

```
// utils/http.ts（关键逻辑示意）

// 请求拦截：自动附加 Token
axios.interceptors.request.use(config => {
  const token = getToken()
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

// 响应拦截：统一错误处理
axios.interceptors.response.use(
  response => response.data,    // 直接返回 data 字段
  error => {
    const status = error.response?.status
    if (status === 401) {
      clearToken()
      router.push('/login')
    } else if (status === 403) {
      ElMessage.error('权限不足，请联系管理员')
    } else if (status >= 500) {
      const traceId = error.response?.data?.trace_id
      ElMessage.error(`服务异常，请稍后重试${traceId ? `（${traceId}）` : ''}`)
    }
    return Promise.reject(error)
  }
)
```

4.4 状态管理设计（Pinia）

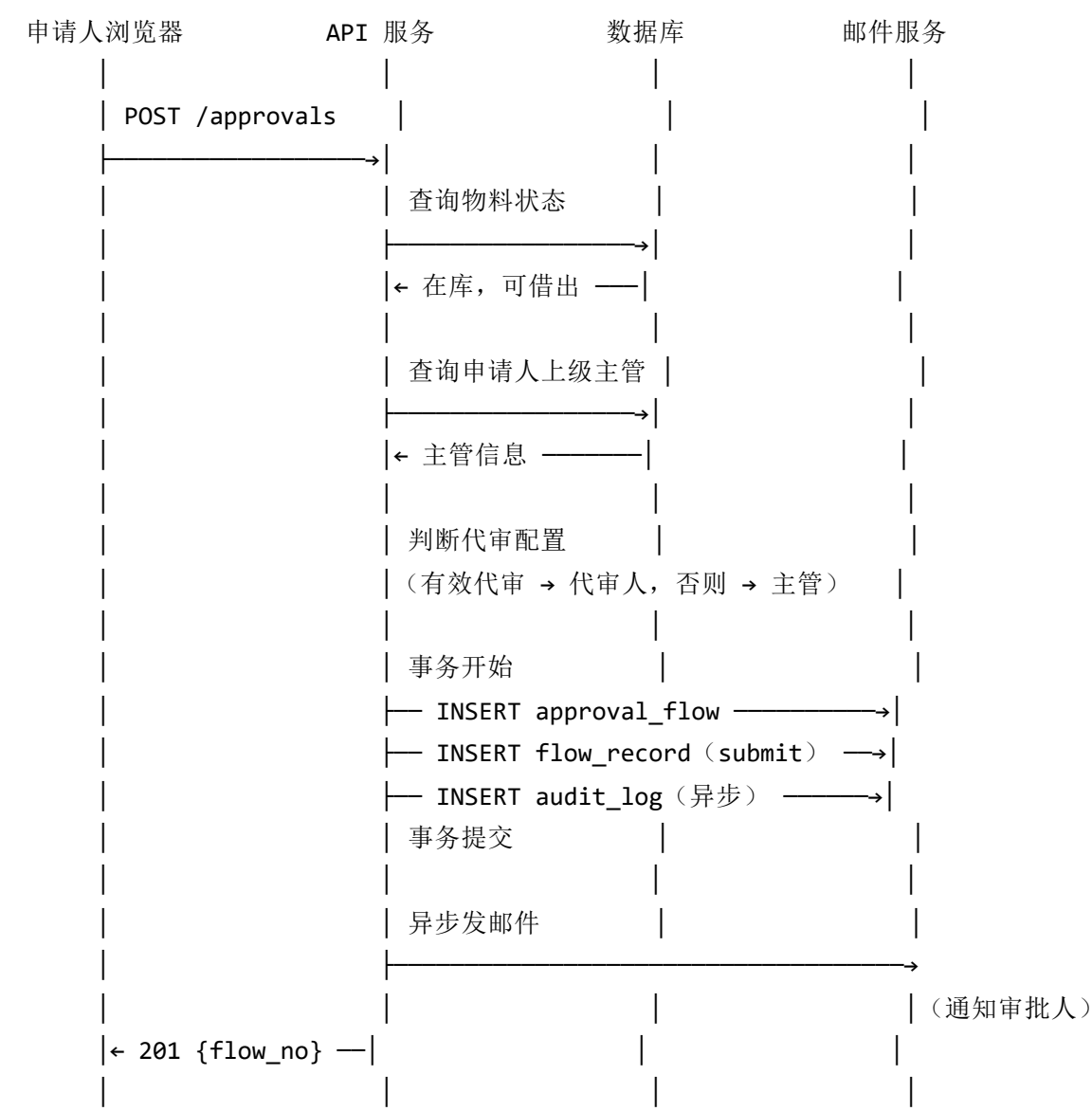
仅对需要跨组件共享的数据使用 Pinia，不过度使用全局状态。

Store	存储内容	更新时机
auth	当前用户信息、JWT Token、角色	登录/注销/Token 刷新
category	类别树数据（只读缓存）	进入需要类别选择的页面时懒加载，管理员修改后失效
location	位置树数据（只读缓存）	同上

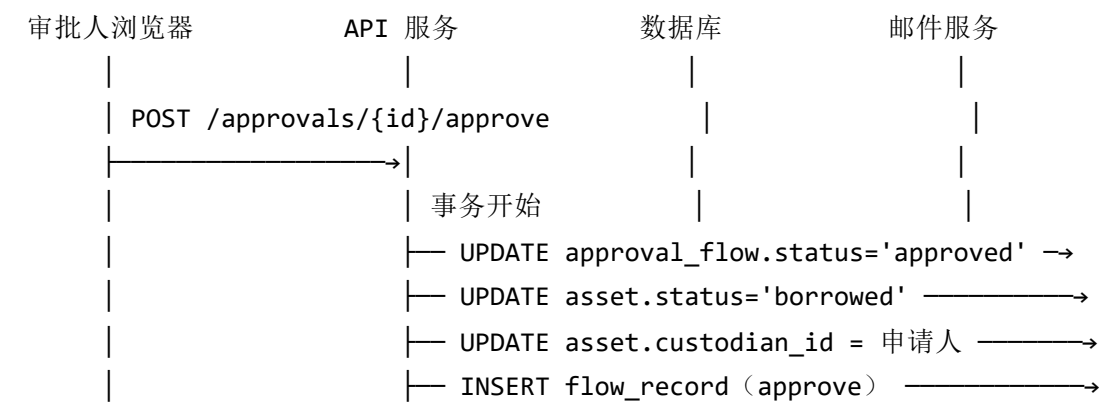
页面内的列表数据、筛选条件、分页状态等**不放入全局 Store**，使用组件内 `ref/reactive` 管理。

5. 数据流与核心时序

5.1 借出申请完整时序

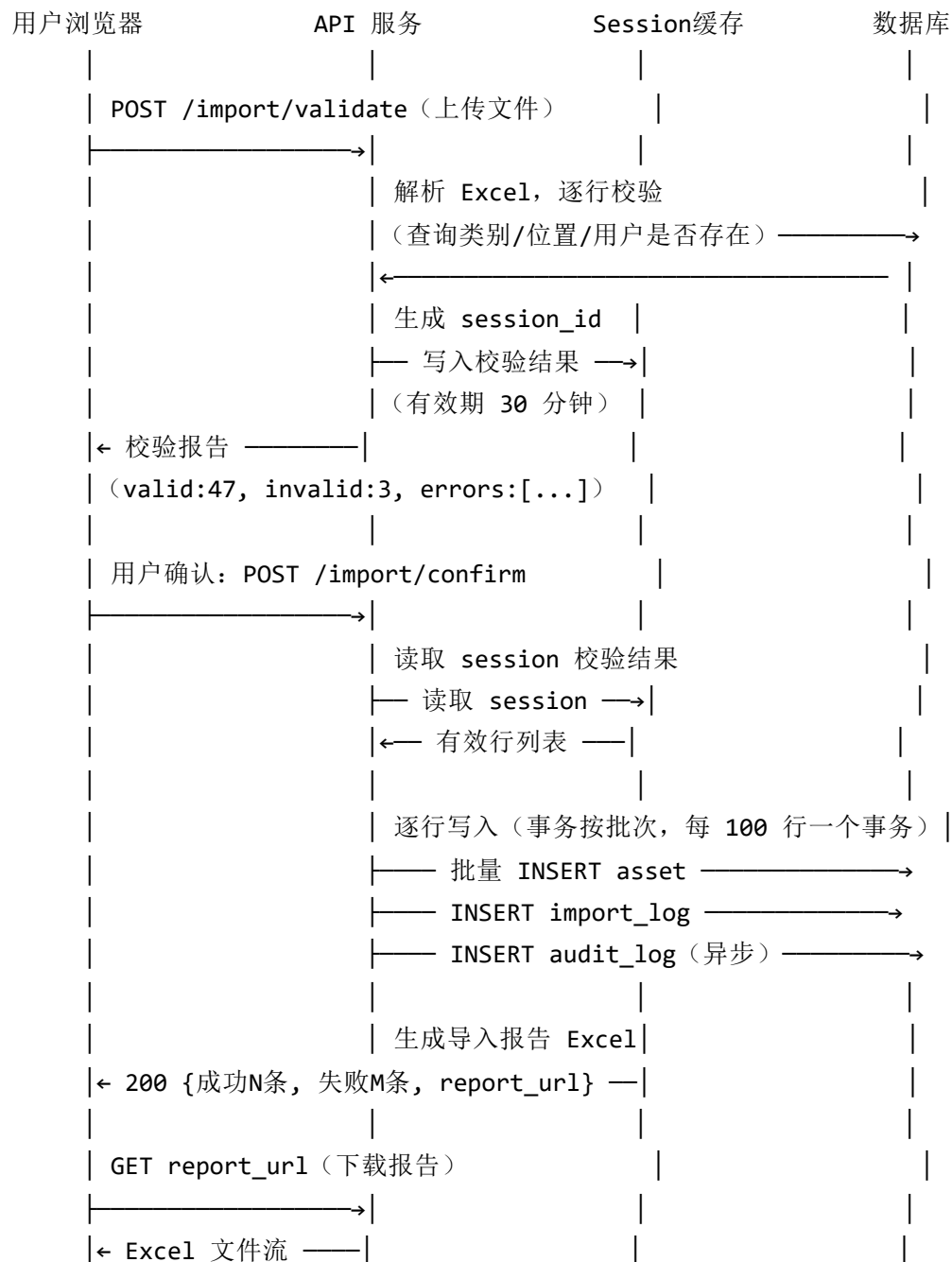


...（审批人登录，查看待审批）...

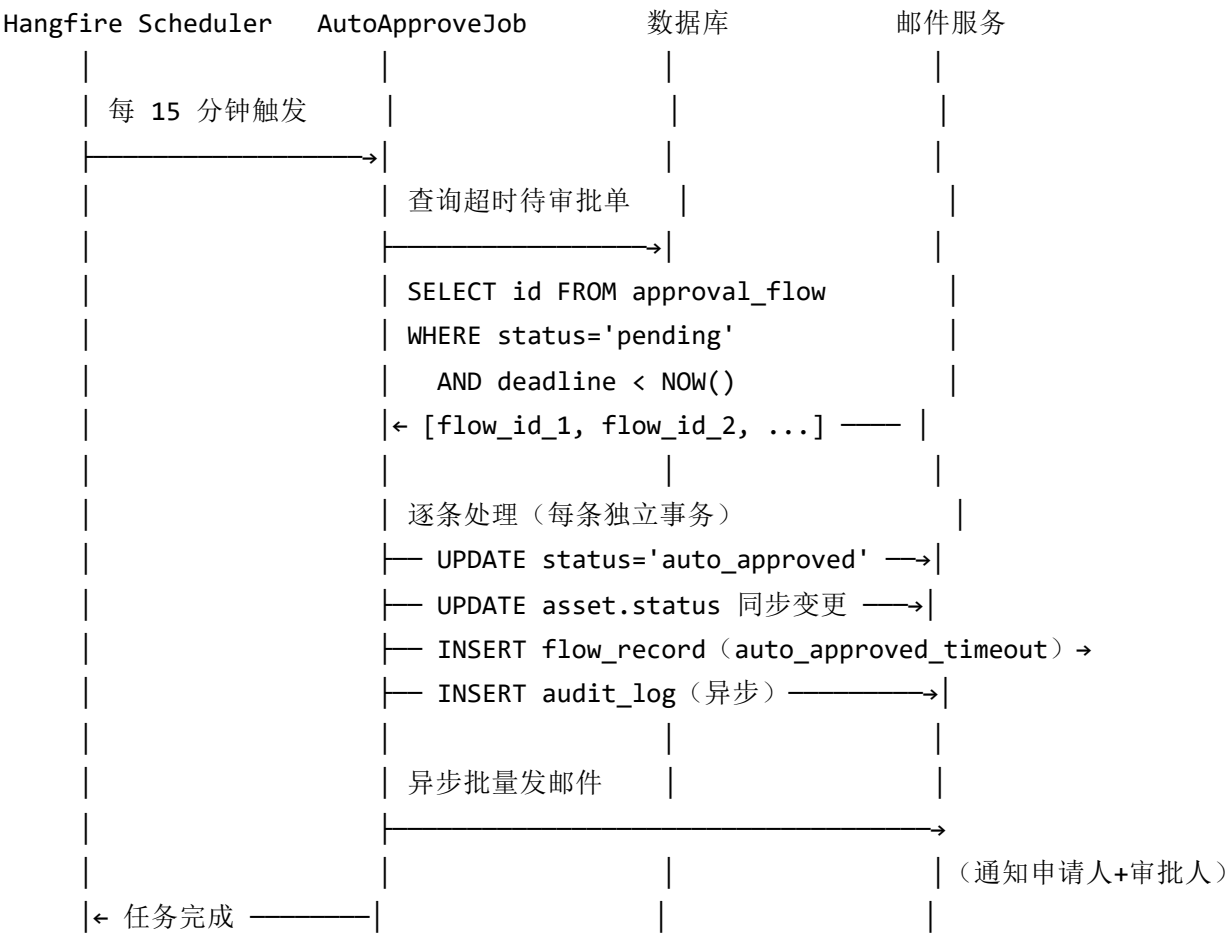




5.2 Excel 导入时序



5.3 超时自动审批时序（Hangfire 定时任务）



6. 关键子系统设计

6.1 编号规则引擎详细设计

接口定义（领域层）：

```
interface INumberGenerator {  
    Task<string> GenerateAsync(int categoryId);  
}
```

实现逻辑（基础设施层）：

1. 读取 `number_rule`（加行锁：SELECT ... WITH (UPDLOCK, ROWLOCK)）
2. 解析 `segment_def` JSON → 各 `Segment` 对象列表
3. 遍历 `Segment` 列表拼接：
 - `FixedSegment` → 直接取 `value`
 - `SeqSegment` → `seq_current + 1`，补零到 `length` 位
4. 乐观更新：

```
UPDATE number_rule  
SET seq_current = @newSeq, updated_at = GETDATE()  
WHERE id = @id AND seq_current = @oldSeq
```
5. 受影响行数 == 0 → 重试（最多 3 次，间隔 50ms）
6. 返回拼接结果

防护机制：

- 编号生成与 `INSERT asset` 在同一数据库事务内
- 事务回滚时 `seq_current` 不回退（编号序列允许空洞，不允许重复）

6.2 邮件通知子系统设计

设计目标：邮件发送失败不影响主流程，支持重试。

接口定义：

```
interface IEmailService {  
    Task SendAsync(EmailMessage message);  
}
```

EmailMessage:

```
{  
    To: string[],  
    Subject: string,  
    Body: string,           // HTML  
    TemplatKey?: string,    // 模板 Key (从 notify_template 表读取)  
    Variables?: Dictionary<string, string> // 模板变量替换  
}
```

实现分两层：

1. MailKitEmailService: 直接 SMTP 发送（底层）
2. EmailDispatcher: 上层调度，发送失败时：
 - 写入 audit_log (action_type=EMAIL_FAIL, summary 含失败原因)
 - 将消息加入 Hangfire 重试队列 (BackgroundJob.Schedule, 延迟 5 分钟重试)
 - 最多重试 3 次，全部失败后不再重试，日志保留

模板变量替换：

- 模板存储在 system_setting / notify_template 表
- 变量格式：{{asset_no}}、{{applicant_name}}、{{deadline}}
- 替换逻辑在 EmailDispatcher 中执行，不依赖模板引擎

6.3 文件存储设计

存储路径规则：

`{UploadRootPath}/{year}/{month}/{guid}{ext}`

示例：`C:\AssetMgmt\uploads\2026\06\3f2a1b9e-xxxx.jpg`

数据库存储：

`asset.attachment_paths` 字段存相对路径 JSON 数组：

`["2026/06/3f2a1b9e-xxxx.jpg", "2026/06/4a1b2c3d-xxxx.docx"]`

访问控制：

`GET /assets/{id}/attachments/{filename}`

→ 后端鉴权通过后，通过 `PhysicalFile()` 返回文件流

→ 不直接暴露物理路径，防止路径遍历攻击

文件大小限制：单文件最大 20MB（在 `appsettings` 中配置）

允许类型：`.jpg .jpeg .png .gif .doc .docx .xls .xlsx`

7. 安全架构设计

7.1 认证流程

登录流程：

1. 客户端 POST /auth/login {employee_no, password}
2. 服务端验证密码 (BCrypt.Verify)
3. 验证通过 → 签发 JWT Token (有效期 8 小时)
Payload: { sub: userId, name, role, exp }
4. 客户端存储 Token (localStorage)
5. 后续请求携带 Authorization: Bearer <token>
6. 服务端中间件验证 Token 有效性和过期时间

Token 续签：

- 剩余有效期 < 1 小时时，响应头携带 X-Refresh-Token: <newToken>
- 客户端 axios 响应拦截器检测此头，自动替换本地 Token
- 用户无感知续签，8 小时内活跃使用不需要重新登录

注销处理：

- 服务端维护 Token 黑名单 (存 Redis 或数据库 token_blacklist 表)
- 注销时将 Token jti 加入黑名单，有效期与 Token 过期时间一致
- 中间件验证时额外检查黑名单

7.2 权限控制

控制粒度：接口级 (不做字段级控制，前端配合角色显示/隐藏)

实现方式：自定义 [RequireRole("admin", "warehouse")] 特性

+ ActionFilter 读取当前用户 role 进行校验

示例：

```
[HttpPost]
[RequireRole("warehouse", "admin")]
public async Task<IActionResult> CreateAsset([FromBody] CreateAssetRequest req)
{ ... }
```

越权防护：

- 普通员工查看审批详情：校验 applicant_id 或 approver_id 是否为当前用户
- 普通员工不可查看他人的申请列表 (接口强制加 applicant_id = currentUserId 条件)
- 附件下载：校验请求用户是否有权访问该物料

7.3 输入安全

SQL 注入：全程使用 EF Core 参数化查询，不拼接 SQL 字符串

XSS 防护：

- 接口返回 JSON，前端 Vue 模板默认 HTML 转义
- 富文本字段（备注、审批意见）长度限制 + 不渲染 HTML

路径遍历：

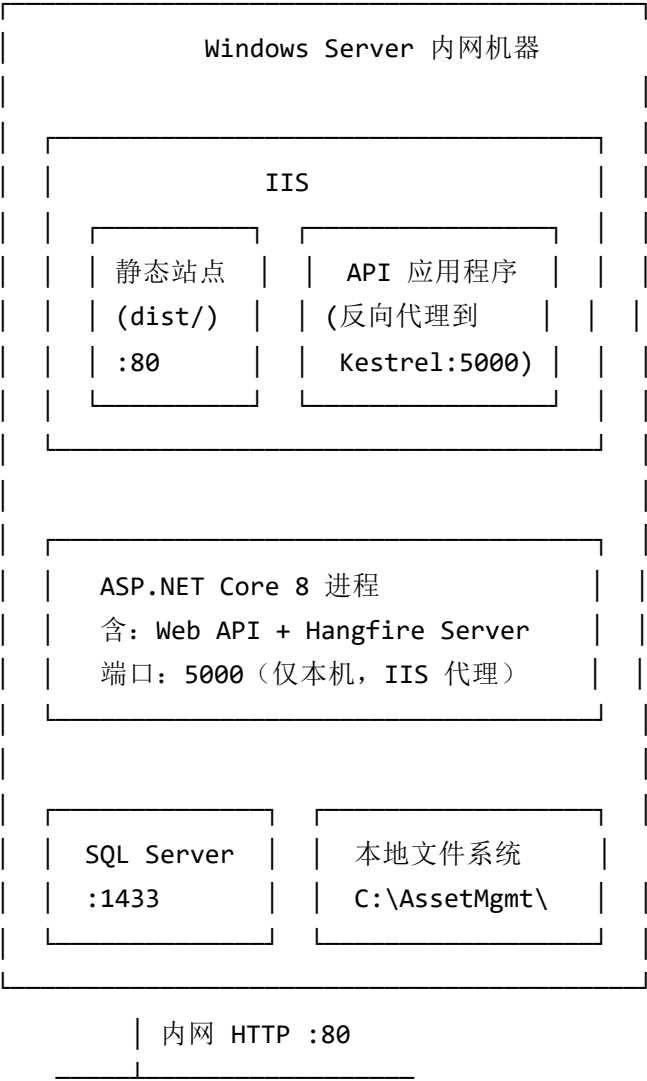
- 文件名使用服务端生成的 GUID，不使用用户上传的原始文件名
- 文件访问接口校验路径在允许目录内

请求体大小限制：

- JSON 请求：默认 1MB
- 文件上传：20MB（单文件）
- Excel 导入：10MB

8. 部署架构

8.1 单服务器内网部署（标准方案）



局域网内所有 PC / 平板

8.2 前端构建与部署

构建前端

`npm run build` # 输出到 `dist/`

部署方式一: IIS 静态站点

将 `dist/` 内容复制到 IIS 站点目录

`vue-router history` 模式需在 `web.config` 添加 URL Rewrite 规则

`web.config` (放置在 `dist/` 根目录)

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<configuration>
```

```
  <system.webServer>
```

```
    <rewrite>
```

```
      <rules>
```

```
        <rule name="SPA Fallback" stopProcessing="true">
```

```
          <match url=".*" />
```

```
          <conditions>
```

```
            <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
```

```
          </conditions>
```

```
          <action type="Rewrite" url="/index.html" />
```

```
        </rule>
```

```
      </rules>
```

```
    </rewrite>
```

```
  </system.webServer>
```

```
</configuration>
```

8.3 后端发布与托管

```
# 发布 (Self-Contained, 目标机无需安装 .NET Runtime)
dotnet publish -c Release -r win-x64 --self-contained \
  -o C:\AssetMgmt\app

# 或依赖运行时版本 (目标机需安装 .NET 8 Runtime, 包体更小)
dotnet publish -c Release -o C:\AssetMgmt\app

# 数据库迁移
dotnet AssetManagement.exe --migrate # 启动时自动迁移
# 或手动执行 DDL 脚本

# IIS 应用程序池设置
# - .NET CLR 版本: 无托管代码
# - 托管管道模式: 集成
# - 进程模型 > 标识: ApplicationPoolIdentity 或指定服务账号
```

8.4 健康检查接口

```
// Program.cs 注册健康检查
builder.Services.AddHealthChecks()
    .AddSqlServer(connectionString, name: "database")
    .AddHangfire(o => o.MinimumAvailableServers = 1, name: "hangfire");

app.MapHealthChecks("/health", new HealthCheckOptions {
    ResponseWriter = UIResponseWriter.WriteHealthCheckUIResponse
});
```

访问 <http://服务器IP/health> 可查看各依赖服务健康状态。

9. 关键技术决策记录

ADR-001: 选择单体架构而非微服务

决策: 采用单体架构。

背景：系统用户 < 100 人，功能模块边界清晰但耦合度自然较高（物料档案是各模块的核心数据源），团队为单人开发。

结论：微服务带来的分布式复杂性（网络分区、数据一致性、服务发现）远超收益。单体在此规模下开发、调试、部署均更高效。若未来需要扩展，单体内部已按分层架构组织，可按模块渐进式拆分。

ADR-002：审计日志使用 AOP 切面而非手动调用

决策：通过 [ASP.NET](#) Core ActionFilter 自动捕获写操作，生成审计日志。

背景：需要对全部写操作记录日志，手动在每个 Service 方法内调用日志写入，侵入性强且容易遗漏。

结论：AOP 切面在不修改业务代码的前提下统一注入日志逻辑，通过约定（HTTP Method + 路由特性）识别操作类型，审计日志写入异步执行不影响主请求性能。副作用：复杂嵌套调用场景需额外处理，但本系统业务流程清晰，此场景极少。

ADR-003：Excel 导入采用"预校验 + 分步确认"而非一次性导入

决策：导入拆分为预校验和确认两步，中间用 Session 缓存校验结果。

背景：一次性导入若部分失败会给用户带来困惑（到底成功了多少条？哪些失败？），且全部回滚损失大。

结论：预校验让用户提前知道哪些行有问题，确认时逐行写入、失败行跳过，最终提供完整报告。Session 有效期 30 分钟，确保用户不会基于过期校验结果提交。

ADR-004：报废使用逻辑删除（is_deleted）而非物理删除

决策：报废物料置 `is_deleted = true`，不删除数据库记录。

背景：物料档案和历史审批记录存在外键关联，物理删除会导致历史记录孤立。报废后档案仍有审计和对账价值。

结论：逻辑删除保留完整历史，查询时默认过滤（`WHERE is_deleted = 0`），需要查历史时去除过滤条件。代价是部分查询需注意加上此过滤条件，通过 EF Core 全局查询过滤器（`HasQueryFilter`）统一处理，不需要每处手动添加。

ADR-005：选择 Hangfire 而非 Windows 任务计划或独立服务

决策：定时任务使用 Hangfire（In-Process 模式，与 Web API 同进程）。

背景：需要定时执行超时审批扫描、逾期提醒、日志清理等任务，需要持久化（重启不丢失）、可监控、支持重试。

结论：Hangfire 满足全部需求，且自带 Dashboard（/hangfire）供运维查看任务历史和失败原因。In-Process 模式无需额外部署独立进程，降低运维复杂度。Windows 任务计划缺乏持久化和重试机制，不适合。

10. 部署与运维

10.1 部署前检查清单

服务器环境检查

- ☐ Windows Server版本：2016及以上
- ☐ .NET 8 Runtime已安装（或使用自包含发布）
- ☐ SQL Server 2017+已安装并配置
- ☐ IIS已安装并启用ASP.NET Core模块
- ☐ 磁盘可用空间：系统盘≥10GB，数据盘≥50GB

网络与端口检查

- ☐ 内网IP地址已分配并固定
- ☐ 80端口（HTTP）可用
- ☐ 5000端口（Kestrel）未被占用
- ☐ 1433端口（SQL Server）可访问
- ☐ SMTP邮件服务器内网可达

数据库准备

- ☐ 创建数据库：AssetManagement
- ☐ 创建数据库用户并授权（db_owner）
- ☐ 配置连接字符串到appsettings.json
- ☐ 执行数据库迁移脚本

应用配置检查

- ☐ appsettings.json配置完整（数据库、邮件、JWT密钥）
- ☐ 附件存储路径已创建（如C:\AssetMgmt\uploads）
- ☐ 日志目录已创建（如C:\AssetMgmt\logs）
- ☐ 初始管理员账号已准备

权限配置

- ☐ IIS应用程序池标识对附件目录有读写权限
- ☐ IIS应用程序池标识对日志目录有写权限
- ☐ SQL Server登录用户有数据库访问权限

10.2 首次部署步骤

1. 发布后端应用

```
dotnet publish -c Release -r win-x64 --self-contained -o C:\AssetMgmt\app
```

2. 配置IIS站点（前端）

- 创建站点：资产管理系统-前端
- 物理路径：C:\AssetMgmt\frontend
- 绑定：http://*:80
- 复制dist/内容到物理路径
- 添加web.config实现SPA路由回退

3. 配置IIS应用程序（后端）

- 创建应用程序池：AssetMgmt（无托管代码）
- 在前端站点下创建应用程序：/api
- 物理路径：C:\AssetMgmt\app
- 应用程序池：AssetMgmt

4. 数据库初始化

```
cd C:\AssetMgmt\app
dotnet AssetManagement.Api.dll --migrate
```

5. 初始化基础数据

- 执行SQL脚本：创建默认管理员账号
- 导入基础数据：类别、位置（可选）

6. 验证部署

- 访问：<http://服务器IP>
- 访问：<http://服务器IP/api/health>
- 登录管理员账号，创建测试资产

10.3 监控方案

应用监控指标

指标	阈值	监控方式	告警
CPU使用率	>80%持续5分钟	Windows性能计数器	发送邮件
内存使用率	>85%	Windows性能计数器	发送邮件
磁盘可用空间	<5GB	定时脚本检查	发送邮件
API响应时间	>3秒	应用性能日志	记录日志
数据库连接失败	连续3次	/health接口	发送邮件
Hangfire任务失败	单个任务连续3次失败	Hangfire Dashboard	发送邮件

业务监控指标

指标	监控频率	监控方式
每日新增资产数	每天	数据库查询
每日审批工单数	每天	数据库查询
逾期未归还资产数	每天	定时任务扫描
超时自动审批数	每天	定时任务统计
邮件发送失败数	每天	audit_logs表统计

日志监控

- **错误日志**：每小时检查，含trace_id便于排查
- **性能日志**：记录慢查询（>1秒），每天分析
- **审计日志**：重点监控敏感操作（删除、报废、权限变更）

10.4 备份与恢复

备份策略

备份对象	频率	保留期	存储位置
SQL Server数据库	每天02:00	30天	内网NAS
附件文件目录	每天02:00	30天	内网NAS
应用配置文件	每次修改后	永久	版本控制

备份脚本示例

```
# backup.bat (Windows计划任务每天02:00执行)
@echo off
set BACKUP_DATE=%date:~0,4%%date:~5,2%%date:~8,2%
set DB_BACKUP=\\nas\backup\db\AssetManagement_%BACKUP_DATE%.bak
set FILE_BACKUP=\\nas\backup\files\uploads_%BACKUP_DATE%

:: 数据库备份
sqlcmd -S localhost -Q "BACKUP DATABASE [AssetManagement] TO DISK='%DB_BACKUP%'"

:: 文件备份
robocopy C:\AssetMgmt\uploads %FILE_BACKUP% /MIR /Z /R:3

:: 清理30天前的备份
forfiles /P \\nas\backup\db /M *.bak /D -30 /C "cmd /c del @path"
forfiles /P \\nas\backup\files /D -30 /C "cmd /c rd /s /q @path"
```

恢复演练（每季度一次）

- 1. 在测试服务器恢复最新备份
- 2. 验证数据完整性
- 3. 验证应用可正常启动和访问
- 4. 记录恢复耗时（RTO目标：1小时内）

10.5 故障排查指南

常见问题速查

问题1：无法访问前端页面

- 检查IIS站点是否启动
- 检查80端口是否被占用： `netstat -ano | findstr :80`
- 检查防火墙规则（内网通常关闭）

问题2：API接口500错误

- 查看日志： `C:\AssetMgmt\logs\`最新日志文件
- 检查trace_id对应的详细错误
- 检查数据库连接字符串是否正确

问题3：数据库连接失败

- 检查SQL Server服务是否运行
- 检查连接字符串：服务器地址、端口、用户名、密码
- 测试连接： `sqlcmd -S localhost -U user -P password`

问题4：邮件发送失败

- 检查SMTP服务器地址和端口
- 检查邮箱账号密码是否正确
- 查看audit_logs表中的EMAIL_FAIL记录

问题5：Hangfire定时任务未执行

- 访问Hangfire Dashboard： <http://服务器IP/api/hangfire>
- 检查任务状态（Succeeded/Failed）
- 查看失败原因和重试记录

问题6：编号生成冲突

- 高并发场景下乐观锁重试3次后仍失败
- 解决：稍后重试，或检查number_rules表seq_current字段

本文档版本 v1.0 (2026-06-10)，架构设计基线，后续变更须更新版本并说明变更原因。